LLM Retrieval Augmented Generation Systems

Introduction. Large language models are increasingly paired with retrieval systems so that answers can be grounded in private, fresh, and domain specific documents. A retrieval augmented generation pipeline usually starts by loading documents, splitting them into meaningful chunks, creating vector embeddings, and storing those vectors in a searchable database. When a user asks a question, the system embeds the question, finds similar chunks, and passes those chunks into an answer generator. This approach reduces hallucination because the model receives relevant evidence instead of relying only on parameters learned during training.

Chunking Strategy. Chunking is one of the most important design choices in an LLM search system. Very large chunks may contain useful context, but they can dilute the semantic signal inside the embedding. Very small chunks may match a query precisely, but they can lose surrounding facts that are required to form a complete answer. Many systems use overlapping windows so that concepts spanning a boundary remain discoverable. Practical chunk sizes often depend on document structure, average paragraph length, token limits, and the expected style of user questions.

Embedding Search. Embeddings transform text into numerical vectors that preserve semantic relationships. A question about transformer attention should be close to passages that explain self attention, even when the exact words are different. Cosine similarity is commonly used to compare the query vector with document vectors. The highest scoring chunks are returned as context. In production systems, approximate nearest neighbor indexes can speed up retrieval across millions of chunks, while smaller prototypes can compute similarity directly from stored vectors.

Caching. Caching improves latency and reduces repeated computation. If many users ask similar or identical questions, the system can reuse previous retrieval results. A cache record may include the normalized query, the number of requested results, the document index version, and the response payload. The cache should be invalidated whenever documents are reindexed, otherwise the user may receive answers based on stale chunks. MongoDB is a convenient cache store because it can hold both structured metadata and response documents.

Evaluation. A retrieval augmented generation system should be evaluated at several levels. Retrieval quality can be measured by checking whether the correct source chunk appears in the top results. Answer quality can be reviewed for faithfulness, coverage, citation accuracy, and refusal behavior when context is insufficient. Good systems make sources visible so that a reader can inspect the evidence. This is especially important for business, legal, medical, and research workflows where the answer must be traceable.

Introduction. Large language models are increasingly paired with retrieval systems so that answers can be grounded in private, fresh, and domain specific documents. A retrieval augmented generation pipeline usually starts by loading documents, splitting them into meaningful chunks, creating vector embeddings, and storing those vectors in a searchable database. When a user asks a question, the system embeds the question, finds similar chunks, and passes those chunks into an answer generator. This approach reduces hallucination because the model receives relevant evidence instead of relying only on parameters learned during training.

Chunking Strategy. Chunking is one of the most important design choices in an LLM search system. Very large chunks may contain useful context, but they can dilute the semantic signal inside the embedding. Very small chunks may match a query precisely, but they can lose surrounding facts that are required to form a complete answer. Many systems use overlapping windows so that concepts spanning a boundary remain discoverable. Practical chunk sizes often depend on document structure, average paragraph length, token limits, and the expected style of user questions.

Embedding Search. Embeddings transform text into numerical vectors that preserve semantic relationships. A question about transformer attention should be close to passages that explain self attention, even when the exact words are different. Cosine similarity is commonly used to compare the query vector with document vectors. The highest scoring chunks are returned as context. In production systems, approximate nearest neighbor indexes can speed up retrieval across millions of chunks, while smaller prototypes can compute similarity directly from stored vectors.

Caching. Caching improves latency and reduces repeated computation. If many users ask similar or identical questions, the system can reuse previous retrieval results. A cache record may include the normalized query, the number of requested results, the document index version, and the response payload. The cache should be invalidated whenever documents are reindexed, otherwise the user may receive answers based on stale chunks. MongoDB is a convenient cache store because it can hold both structured metadata and response documents.

Evaluation. A retrieval augmented generation system should be evaluated at several levels. Retrieval quality can be measured by checking whether the correct source chunk appears in the top results. Answer quality can be reviewed for faithfulness, coverage, citation accuracy, and refusal behavior when context is insufficient. Good systems make sources visible so that a reader can inspect the evidence. This is especially important for business, legal, medical, and research workflows where the answer must be traceable.

Introduction. Large language models are increasingly paired with retrieval systems so that answers can be grounded in private, fresh, and domain specific documents. A retrieval augmented generation pipeline usually starts by loading documents, splitting them into meaningful chunks, creating vector embeddings, and storing those vectors in a searchable database. When a user asks a question, the system embeds the question, finds similar chunks, and passes those chunks into an answer generator. This approach reduces hallucination because the model receives relevant evidence instead of relying only on parameters learned during training.

Chunking Strategy. Chunking is one of the most important design choices in an LLM search system. Very large chunks may contain useful context, but they can dilute the semantic signal inside the embedding. Very small chunks may match a query precisely, but they can lose surrounding facts that are required to form a complete answer. Many systems use overlapping windows so that concepts spanning a boundary remain discoverable. Practical chunk sizes often depend on document structure, average paragraph length, token limits, and the expected style of user questions.

Embedding Search. Embeddings transform text into numerical vectors that preserve semantic relationships. A question about transformer attention should be close to passages that explain self attention, even when the exact words are different. Cosine similarity is commonly used to compare the query vector with document vectors. The highest scoring chunks are returned as context. In production systems, approximate nearest neighbor indexes can speed up retrieval across millions of chunks, while smaller prototypes can compute similarity directly from stored vectors.

Caching. Caching improves latency and reduces repeated computation. If many users ask similar or identical questions, the system can reuse previous retrieval results. A cache record may include the normalized query, the number of requested results, the document index version, and the response payload. The cache should be invalidated whenever documents are reindexed, otherwise the user may receive answers based on stale chunks. MongoDB is a convenient cache store because it can hold both structured metadata and response documents.